F42: Fortran coding guides

Asis Hallab, "el Bobito"

May 6, 2025

1 Memory Passing Behavior in Fortran (F42 Standard for Tensor Omics)

1.1 General Principles

In Fortran (standards 77–2018), arguments to subroutines and functions are, by default, passed **by reference** rather than by copying. This applies to scalars, arrays, and derived types.

Thus:

- Scalars (e.g., `integer`, `real`) are passed by reference.
- Arrays (e.g., `integer, dimension(:)`, `real, dimension(:,:)`) are passed by reference.
- Derived types are passed by reference.

1.2 When Copies Are Made

Despite the default pass-by-reference behavior, hidden copies may be generated by the compiler in the following situations:

- **Passing expressions**: e.g., `call process(a+b)` requires evaluation and temporary storage.
- **Passing strided array sections**: e.g., `array(1:10:2)` is non-contiguous and may trigger copying.
- **Passing non-contiguous slices**: particularly relevant for rows in a 2D array.

1.3 Tensor Omics DataTables: Column and Row Passing

Tensor Omics DataTables store typed data in 2D Fortran arrays of the form:

- `real_array(n_rows, n_real_columns)`
- `int_array(n_rows, n_int_columns)`
- `char_array(n_rows, n_char_columns)`

Accessing Columns:

- Example: `real_array(:, j)`
- Behavior: Passed by reference, **no copy made**.
- Reason: Slicing across the first index in column-major layout yields contiguous memory.

Accessing Rows:

- Example: `real_array(i, :)`
- Behavior: Typically triggers a hidden copy.
- Reason: Slicing across the second index accesses non-contiguous memory addresses, requiring packing into a contiguous temporary.

1.4 Summary Tables

1.4.1 General Fortran Behavior

Action	Example	Copy Made?
Pass full array	`array`	No
Pass contiguous slice	`array(:, j)`	No
Pass strided slice	`array(1:10:2, j)`	Yes
Pass expression	`array(:, j) * 2.0`	Yes

1.4.2 Tensor Omics DataTables Specifics

Access Type	Example	Copy Made?
Access full column	`real_array(:, j)`	No
Access full row	`real_array(i, :)`	Yes

1.5 Best Practice Recommendation for Tensor Omics

- Always design core functions to work with columns if possible.
- Avoid passing strided slices or expressions directly to subroutines.
- Treat rows carefully: copy manually if absolutely needed, otherwise prefer per-element processing.

1.6 Conclusion

Efficient memory handling is critical for Tensor Omics. Understanding Fortran's passing conventions allows maximization of speed, minimization of hidden temporaries, and preservation of F42 computational standards.

2 Critical Practices in TOX Interfacing and Control Flow

This section consolidates three key implementation standards for TOX and F42 compliance: correct R–Fortran interfacing, prohibition of `GOTO`, and manual memory cleanup.

2.1 Why `bind(C)` Is Dangerous for R Interfacing

Using `bind(C)` in Fortran changes the function's symbol name and calling convention to conform to the C ABI. This is incompatible with R's native `.Fortran()` interface, which relies on legacy Fortran symbol conventions (e.g., appending underscores, pass-by-address semantics).

Problems Caused by `bind(C)` in R Packages

- **Breaks Symbol Lookup:** R expects mangled names like `mysub_`, which `bind(C)` suppresses.
- **Incorrect Argument Handling:** Changes to calling convention may result in wrong memory alignment or crashes.
- **Unnecessary:** R handles native Fortran linking automatically during package build.

Correct Way to Interface R with Fortran

1. Write your subroutines in pure Fortran *without* `bind(C)`.
2. Place Fortran files in `src/` directory of the package.
3. Register your compiled code using `useDynLib(pkgname)` in `NAMESPACE`.
4. Call Fortran code from R via `.Fortran()`:

```
result <- .Fortran("mysub",
  as.double(x), as.double(y), result = double(1))$result
```

Minimal Example of Package Setup

- `DESCRIPTION`: no special flags needed.
- `NAMESPACE`: `useDynLib(mytoxpkg)`
- `src/mysub.f90`:

```
subroutine mysub(x, y, result)
  real(8) :: x, y, result
  result = x + y
end subroutine
```

2.2 Why `GOTO` Is Forbidden in F42 and TOX

`GOTO` is a low-level control structure that violates the principles of structured, readable, and verifiable code. While allowed by Fortran, its use is incompatible with both modern software engineering and WebAssembly (WASM) compilation.

Reasons for Prohibition

- **Non-Structured Control Flow:** Makes code hard to reason about and maintain.
- **Incompatible With WASM:** WebAssembly requires structured control graphs.
- **Opaque for Debugging:** Introduces implicit jump logic that hinders traceability.
- **Fails F42 Goals:** Violates statelessness, modularity, and verifiability.

2.3 How to Replace `GOTO` Correctly

1. Use `exit` to break out of a loop

```
do i = 1, n
  if (array(i) < 0.0) then
    exit
  end if
end do
```

2. Use `cycle` to skip to next iteration

```
do i = 1, n
  if (.not. valid(i)) cycle
  call process(i)
end do
```

3. Use `select case` for structured branching

```
select case(code)
case(1)
  call handle_case1()
case(2)
  call handle_case2()
```

```
case default
  call handle_default()
end select
```

2.4 Memory Cleanup: Manual Destructors

Since Fortran has no `try/finally` construct and F42 disallows `final ::` destructors due to their opacity, all memory cleanup must be performed manually.

Pattern: Explicit Cleanup Subroutine

```
subroutine deallocate_datatable(dt)
  type(DataTable), intent(inout) :: dt

  if (allocated(dt%real_array)) deallocate(dt%real_array)
  if (allocated(dt%char_array)) deallocate(dt%char_array)
  if (allocated(dt%int_array))  deallocate(dt%int_array)
  if (allocated(dt%column_names)) deallocate(dt%column_names)
end subroutine
```

Pattern: Simulated try-finally via logical block

```
logical :: success
success = .false.

call allocate_datatable(dt, ierr)
if (ierr == 0) then
  call compute(dt)
  success = .true.
end if

call deallocate_datatable(dt)
```

Conclusion

All R–Fortran interfacing, control flow, and memory management in TOX must follow F42 principles:

- Never use `bind(C)` unless wrapping via `.Call()` (not `.Fortran()`).
- Never use `GOTO` — always use structured constructs.
- Always provide manual cleanup subroutines for types containing allocatables.

These principles ensure safety, readability, WASM compatibility, and long-term maintainability.

3 Unified Parallel Loop Snippet

To streamline parallel execution across different computing environments, we define a unified Fortran snippet that automatically generates precompiler code enabling compilation to one of three execution modes: **Coarrays**, **OpenMP**, or **serial fallback**. This is achieved using preprocessor directives that adapt the loop structure based on compiler flags. The loop body itself remains unchanged, ensuring that the parallelization mechanism is orthogonal to the logic being executed.

Below is the result of expanding the `f42:do-parallel` snippet, which selects the appropriate backend at compile time:

```
! ----------------------
! START OF PARALLEL LOOP
! ----------------------
# ifdef USE_COARRAY
start_i = (this_image() - 1) * N / num_images() + 1
end_i   = min(this_image() * N / num_images(), N)

do i = start_i, end_i
# endif

# ifdef USE_OPENMP
!$omp parallel do private(i)
do i = 1, N
# endif

#if !defined(USE_COARRAY) && !defined(USE_OPENMP)
do i = 1, N
#endif

! BODY OF THE PARALLEL LOOP
! -----------------------------

! -----------------------------
! END BODY OF THE PARALLEL LOOP

end do ! END OF DO-LOOP

# ifdef USE_COARRAY
sync all
# endif

# ifdef USE_OPENMP
!$OMP END PARALLEL
# endif

! --------------------
! END OF PARALLEL LOOP
! --------------------
```

Compile with:

- `-DUSE_OPENMP` and `-fopenmp` for OpenMP
- `-DUSE_COARRAY` for Coarrays (e.g. with `-fcoarray=single` for shared memory, or `-fcoarray=lib` for distributed)

3.1 Guideline: Use `f42:do-parallel` only in "Root Loops"

To ensure clarity, efficiency, and robustness of parallel execution in Tensor Omics, parallel loops using the `f42:do-parallel` snippet should only be used at the *root level* of computational routines—that is, in the outermost loops directly executed within a parallelized function or subroutine. These *root-loops* are not nested inside other loops or deeply obscured by function calls.

This restriction avoids unnecessary overhead, data locality issues, and potential thread contention that can arise when parallelization is applied within inner or nested computation layers. Keeping parallelization

explicit and confined to root-loops ensures predictable control over memory access and parallel execution, aligning with the minimalist and efficient design principles of F42. Developers are encouraged to refactor inner computations into pure, stateless subroutines and apply parallelization only where it governs the outermost iteration logic.